

Papers Dock



PAPER 4

PRACTICE PAPER 1

Open the document **evidence.doc**.

Make sure that your name, centre number and candidate number will appear on every page of this document. This document will contain your answers to each question.

Save this evidence document in your work area as:

evidence_ followed by your centre number_candidate number, for example:
evidence_zz999_9999

A class declaration can be used to declare a record.

A list is an alternative to an array.

1 A program stores the following 7 names in a 1D array with identifier nameData.

“Farjad” “Khadija” “Ali” “Mazin” “Taha” “Fakhir” “Zainab”

(a) Write program code for a **new program** to:

- declare the global 1D array, nameData, with 7 elements
- initialise nameData in the main program using the data values shown

Save your program as **question1**.

Copy and paste the program code into part 1(a) in the evidence document.

[2]

(b) (i) A function, linearSearch(), takes a string as a parameter and performs a linear search on nameData to find the parameter value. It returns True if it was found and False if it was not found.

Write program code for the function linearSearch()

Save your program.

Copy and paste the program code into part 1(b) in the evidence document.

[6]

(ii) Edit the main program to:

- allow the user to input a string value
- pass the value to `linearSearch()` as the parameter
- output an appropriate message to tell the user whether the search value was found or not.

Save your program.

Copy and paste the program code into part 1(b)(ii) in the evidence document.

[4]

(iii) Test your program with one value that is in the array and one value that is not in the array.

Take a screenshot to show the result of each test.

Save your program.

Copy and paste the screenshots into part 1(b)(iii) in the evidence document.

[2]

(c) Write program code for the procedure `insertionSort ()` to sort the data in alphabetical order

Save your program.

Copy and paste the program code into part 1(b)(ii) in the evidence document.

[6]

(d) Test your program by calling the function `insertion sort`.

Take a screenshot to show the result

Save your program.

Copy and paste the screenshots into part 1(d) in the evidence document.

[1]

2 A treasure box is hidden within a computer game.

The box has a code that needs to be entered to allow the user into the box. The box contains up to 10 objects that are defined as being of the class `FieldObject`. The definition for the class `Box` is:

Box	
Size : STRING Contents : ARRAY[0 : 9] OF FieldObject Lock : STRING Strength : INTEGER	// small, medium or large // the 10 items the box holds // the code to unlock the box // the strength of the box // decreases by 1 each time an // incorrect code is entered
Constructor() Unlock() GetContents() SetSize() SetContents() SetLock() SetStrength()	// instantiates an object of the Box // class and assigns initial values // to the attributes // checks if the code is correct to // unlock the box // returns the array // sets the size of the box // sets the contents of the box // sets the lock code // sets the strength

- (a) The constructor creates a new instance of a box. It takes the size of the box, one item of content and the lock code as parameters. The strength is initialised to 100.

Write **program code** to create the class and constructor for `Box`.

Do **not** write the code for `Unlock()` or any of the set or get methods. Use the constructor for your chosen language.

Save your program.

Copy and paste the program code into part 2(a) in the evidence document.

- (b) The player inputs the code to unlock the box. Each time they enter an incorrect code, the strength of the box decreases by 1. If the strength of the box becomes 0, the box automatically unlocks.

The class `Box` has a method `Unlock()` that:

- takes the code entered as a parameter and checks if it matches the code to unlock the box
- returns `TRUE` if the parameter matches the unlock code
- subtracts 1 from `Strength` if the parameter does not match the unlock code
- checks if the new value of `Strength` is less than 1
- returns `TRUE` if the new value of `Strength` is less than 1, otherwise it returns `FALSE`.

Write **program code** for the method `Unlock()`.

Save your program.

Copy and paste the program code into part 2(b) in the evidence document.

[5]

- (c) The player might need to access the information of the object and might have to update the values in future.

- (i) The class `Box` has a method **`getContents()`** that returns the `Content` array

Write the method `getContents()`

Save your program.

Copy and paste the program code into part 2(c)(i) in the evidence document.

[1]

- (ii) The class `Box` also has setters which updates the values of attributes

Write the methods for `setSize()` `setContents()` `setLock()` `setStrength()`

Save your program.

Copy and paste the program code into part 2(c)(i) in the evidence document.

[4]

3) An unordered linked list uses a 1D array to store the data. Each item in the linked list is of a record type, node, with a field data and a field nextNode.

The current contents of the linked list are:

startPointer	0	Index	data	nextNode
		0	1	1
emptyList	5	1	5	4
		2	6	7
		3	7	-1
		4	2	2
		5	0	6
		6	0	8
		7	56	3
		8	0	9
		9	0	-1

(a) The following is pseudocode for the record type *node*.

```
TYPE node
    DECLARE data : INTEGER
    DECLARE nextNode : INTEGER
ENDTYPE
```

Write program code to declare the record type *node*.

Save your program.

Copy and paste the program code into part 3(a) in the evidence document.

(b) Write program code for the main program.

Declare a 1D array of type node with the identifier linkedList, and initialise it with the data shown in the table on **page 2**.

Declare the pointers

Save your program.

Copy and paste the program code into part 3(b) in the evidence document.

[4]

(c) The procedure **outputNodes()** takes the array and startPointer as parameters. The procedure outputs the data from the linked list by following the nextNode values.

(i) Write program code for the procedure **outputNodes()**.

Save your program.

Copy and paste the program code into part 3(c)(i) in the evidence document.

[6]

(ii) Edit the main program to call the procedure outputNodes().

Take a screenshot to show the output of the procedure outputNodes().

Save your program.

Copy and paste the Screen Shot into part 3(c)(ii) in the evidence document.

[1]

(d) The function, DeleteNode(), takes the linked list and pointers and the value to be removed as parameters.

Updates the pointers and returns True if it was Deleted successfully. If the data to be removed is not in the List then returns False.

(i) Write program code for the function DeleteNode()

Save your program.

Copy and paste the program code into part 3(d)(i) in the evidence document.

[7]

(ii) Edit the main program to:

- call DeleteNode()
- output an appropriate message depending on the result returned from DeleteNode()
- call outputNodes() twice; once before calling DeleteNode() and once after calling DeleteNode().

Save your program.

Copy and paste the program code into part 3(d)(ii) in the evidence document.

[3]

(iii) Test your program by Deleting the data value 56 and take a screenshot to show the output.

Save your program.

Copy and paste the screenshots into part 3(d)(iii) in the evidence document.

[1]

